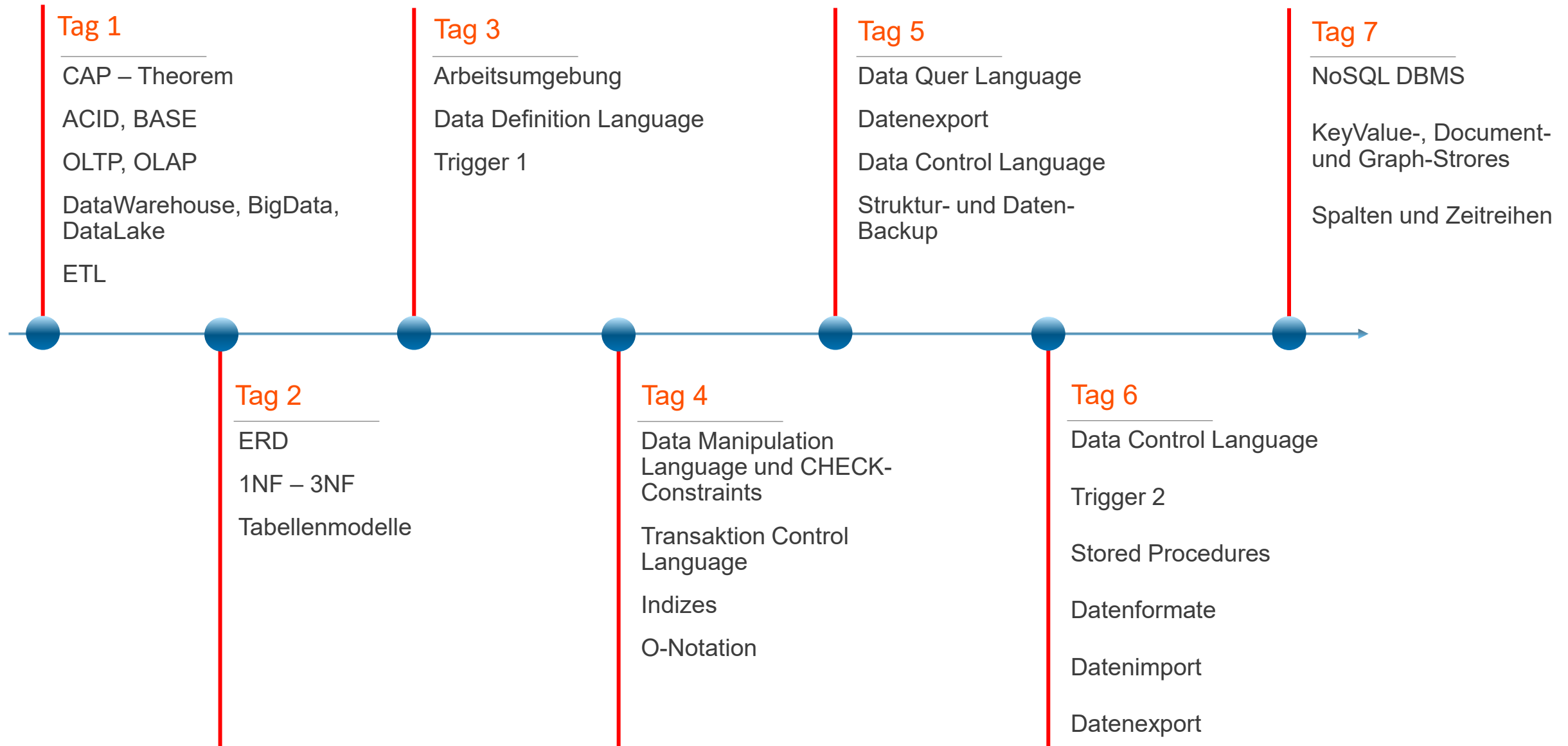


DB2



1

2

3

4

NoSQL Datenbanken

Moderne Mitbewerber
zur Datenhaltung in
verteilten Umgebungen



NoSQL Datenbanken

Moderne Mitbewerber zur Datenhaltung in verteilten Umgebungen

Sinnhaftigkeit von NoSQL

Für einen Fachinformatiker ist das Verständnis von NoSQL (Not Only SQL) essenziell, da es den Werkzeugkasten über die klassische relationale Welt hinaus erheblich erweitert, besonders wenn es um Skalierbarkeit und flexible Datenstrukturen geht.

1. Warum NoSQL?

Bis weit in die 2000er Jahre hinein war die Welt einfach: Daten gehörten in Tabellen, wurden mit SQL abgefragt und folgten den ACID-Prinzipien (Atomicity, Consistency, Isolation, Durability).

Mit dem Aufkommen des Web 2.0 und dem massiven Anstieg der weniger und nicht strukturierten Datenmengen (Social Media, Big Data, Cloud Computing) entstanden jedoch drei massive Probleme für klassische RDBMS:

A. Das Skalierungsproblem (Scalability)

Relationale Datenbanken skalieren primär vertikal (Scale-up): Man kauft einen größeren Server mit mehr RAM und CPU. Das ist teuer und hat physikalische Grenzen. NoSQL wurde für die horizontale Skalierung (Scale-out) entwickelt – man verteilt die Last einfach auf viele günstige Standard-Server.

B. Starre Schemata vs. Flexibilität

In einem RDBMS muss das Schema (die Tabellenstruktur) vorher genau definiert sein. In der modernen App-Entwicklung ändern sich Datenstrukturen aber ständig. NoSQL-Datenbanken sind oft schema-frei (schemaless), was eine agilere Entwicklung ermöglicht.

C. Die Geschwindigkeit (Performance/Throughput)

Bei Millionen von gleichzeitigen Lese- und Schreibzugriffen (z.B. X-Feeds oder Amazon-Warenkörbe) wird der Overhead durch komplexe Tabellen-Joins und ACID-Garantien zum Flaschenhals.

2. Die Entwicklung und das CAP-Theorem

Ein wichtiger Meilenstein für das Verständnis ist das CAP-Theorem. Es besagt, dass ein verteiltes System niemals alle drei der folgenden Eigenschaften gleichzeitig maximieren kann:

Consistency (Konsistenz):

Alle Nutzer sehen zur gleichen Zeit dieselben Daten.

Availability (Verfügbarkeit):

Jede Anfrage wird beantwortet (auch wenn ein Knoten ausfällt).

Partition Tolerance (Ausfalltoleranz):

Das System arbeitet weiter, auch wenn die Kommunikation zwischen Servern unterbrochen ist.

In der NoSQL-Welt entscheidet man sich meist für **P** und muss dann zwischen **C** oder **A** abwägen.

2

Datenbank-Typen

KeyValue-, Document- und Graph-Stores

Einordnung der Datenbank-Typen

NoSQL ist kein einzelnes Produkt, sondern ein Sammelbegriff für vier Hauptkategorien, die wir in dieser Reihe genauer anschauen werden:

Typ	Kurzbeschreibung	Beispiel-Systeme
Document Stores	Speichert Daten als Dokumente (meist JSON/ BSON)	MongoDB, CouchDB
Key-Value Stores	Extrem schnell, ordnet einem Schlüssel einen Wert zu (Key-Value Pair).	Redis, Riak
Column-Family Stores	Spaltenorientiert, optimiert für riesige Datenmengen auf vielen Servern.	Cassandra, HBase
Graph Databases	Fokus auf Beziehungen zwischen Datenpunkten	Neo4j, ArangoDB

Außerdem werden wir auch einen Blick auf Zeitreihen-Datenbanken (Time Series) werfen. Diese sind eine spezialisierte Art von Datenbanken für chronologische Daten (z.B. Sensordaten, Aktienkurse) darstellen.

Dann tauchen wir direkt in die Welt der spaltenorientierten Datenbanken (Column-Family Stores) ein. Um den Unterschied zu verstehen, müssen wir uns zuerst anschauen, wie eine klassische Datenbank (wie MariaDB oder PostgreSQL) "denkt".

Die zeilenorientierte (Row-based)

In der klassischen Welt (RDBMS) werden in einer normalen Datenbank die Daten zeilenweise auf der Festplatte gespeichert. Dazu folgendes Szenario: Stell dir eine Tabelle mit Verkaufsdaten vor: ID, Datum, Produkt, Preis, Filiale.

- Speicherlayout: [1, 2024-01-01, Laptop, 1200€, Berlin], [2, 2024-01-02, Maus, 25€, Hamburg], ...
- Vorteil: Super, wenn du einen kompletten Datensatz (z.B. User-Profil) laden oder ändern willst.
- Nachteil: Wenn du wissen willst: "Wie hoch war der Gesamtumsatz?", muss die Datenbank trotzdem die komplette Zeile lesen, obwohl sie nur die Spalte "Preis" braucht. Das verschwendet extrem viel I/O-Leistung.

Die spaltenorientierte Welt (Column-based)

Hier werden alle Werte einer Spalte direkt hintereinander gespeichert.

Speicherlayout:

- Spalte ID: 1, 2, 3...
- Spalte Preis: 1200, 25, 45...
- Spalte Filiale: Berlin, Hamburg, Berlin...

Der Clou: Wenn man jetzt die Summe der Preise berechnest, springt der Lesekopf der Festplatte (oder der SSD-Controller) nur zu dem Block, in dem die Preise liegen, und liest diese in einem Durchgang ein.

Hierdurch können OLAP-Abfragen 10- bis 100-mal schneller ausgeführt werden und die I/O-Last verringert sich durch die optimalere Anordnung der Daten um bis zu 95%

Die Vorteile in der Praxis (Big Data & Analytics) und der Nutzen für das Data Warehousing oder Business Intelligence.

Hohe Kompressionsrate

Da in einer Spalte oft ähnliche Daten stehen (z.B. viele "Berlin"-Einträge in der Filial-Spalte), lassen sich diese Daten extrem effizient komprimieren. Das spart massiv Speicherplatz (Kompressionsraten von bis zu 70%).

Aggregationen in Millisekundenbereich

Berechnungen wie SUM, AVG oder COUNT über Milliarden von Zeilen sind in Spaltendatenbanken um ein Vielfaches schneller (etwa 5- bis 7-mal schneller).

Skalierbarkeit

Skalierbarkeit: Systeme wie Apache Cassandra oder HBase nutzen "Column Families", innerhalb von Wide-Column-Stores. Daten bzw. Abfragen werden hierbei über viele Server (Knoten) verteilt und die Ergebnisse dann zusammen geführt.

Exkurs: Zeitreihen-Datenbanken (Time Series DBs)

Zeitreihen-Datenbanken (TSDB) sind eine spezialisierte Unterform, die oft spaltenorientierte Konzepte nutzt, aber für einen ganz bestimmten Datentyp optimiert ist: Daten mit Zeitstempel. Typische Merkmale:

Append-only

Daten werden fast nur hinzugefügt, selten geändert. (Ein Temperaturwert von gestern um 12:00 Uhr ändert sich nicht mehr).

Hoher Schreibdruck

Sensoren (IoT), Server-Metriken (CPU-Last) oder Aktienkurse liefern tausende Werte pro Sekunde.

Automatische Aggregation

Eine TSDB kann alte Daten automatisch "zusammenfassen" (z.B. nach 30 Tagen werden aus Sekunden-Werten Stunden-Durchschnitte gemacht, um Platz zu sparen – man nennt das Downsampling).

Bekannte Vertreter:

InfluxDB, Prometheus, TimescaleDB. Häufig wird der so genannte TIG-Stack eingesetzt. Dieser besteht aus den Tools Telegraf (Agent mit Plugins für über 300 Datenquellen), InfluxDB (Zeitreihen-DB) und Grafana (Datenvisualisierung)

Was kann zusammengefasst hinsichtlich spaltenorientierter- und Zeitreihen-Datenbanken gesagt werden

Merkmal	Spalteorientiert (z.B. Cassandra)	Zeitreihen (z.B. InfluxDB)
Hauptfokus	Schnelle Analyse großer Spaltenmengen	Monitoring, IoT, Metriken
Struktur	Column Families / Spalten-Blöcke	Zeitstempel + Messwert + Tags
Anwendungsfall	Big Data Analytics, Reports	Überwachungssysteme, Log-Analyse

Schauen wir uns den Kontrast zwischen der klassischen relationalen Welt (SQL) und der Welt der dokumentenorientierten Datenbanken (NoSQL) an. Als Beispiel nehmen wir ein typisches Szenario: Ein Online-Shop mit Kunden und ihren Bestellungen.

1. Die relationale Struktur (SQL)

In einem RDBMS wie MariaDB müssen wir die Daten normalisieren, um Redundanzen zu vermeiden. Das bedeutet, wir teilen die Informationen auf mehrere Tabellen auf und verknüpfen sie über Fremdschlüssel.

Tabellenstruktur:

Tabelle_Kunden: ID, Vorname, Nachname, Email

Tabelle_Adressen: ID, Kunden_ID, Strasse, PLZ, Stadt

Tabelle_Bestellungen: ID, Kunden_ID, Datum, Summe

Die Abfrage:

Um alle Infos zu einem Kunden zu bekommen, muss man einen JOIN über drei Tabellen machen. Das kostet Rechenleistung, je mehr Daten vorhanden sind.

```
1 SELECT *
2 FROM kunden k
3 JOIN adressen a ON k.id = a.kunden_id
4 JOIN bestellungen b ON k.id = b.kunden_id
5 WHERE k.id = 42;
```

2. Die dokumentenorientierte Struktur (NoSQL, MongoDB)

In einer Dokumentendatenbank wie MongoDB speichert man die Daten so, wie die Applikation sie braucht: als ein zusammenhängendes Objekt (meist im JSON- oder BSON-Format). Man nennt das Denormalisierung.

Das Dokument:

Statt drei Tabellen haben wir ein einziges Dokument für den Kunden "Max Mustermann".

```
1  {
2    "_id": "42",
3    "vorname": "Max",
4    "nachname": "Mustermann",
5    "email": "max@example.com",
6    "anschrift": {
7      "strasse": "Hauptstraße 1",
8      "plz": "12345",
9      "stadt": "Berlin"
10   },
11   "bestellungen": [
12     { "datum": "2024-01-10", "summe": 59.99 },
13     { "datum": "2024-02-15", "summe": 12.50 }
14   ]
15 }
```

Unterschiede SQL – NoSQL Document Store

Hier sind die Hauptunterschiede:

Merkmal	Relational (SQL)	Dokumentenorientiert (NoSQL)
Dateneinheit	Zeile (Row)	Dokument (Document)
Schema	Starr (Fixed Schema)	Flexibel (Dynamic Schema)
Beziehungen	JOINS (Tabellenverknüpfung)	Embedding (Einbetten im Dokument)
Skalierung	Vertikal (stärkerer Server)	Horizontal (mehr Server/ Sharding)
Transaktionen	ACID (sehr streng)	BASE (Eventual Consistency)

Warum nutzt man Dokumente?

Entwicklerfreundlich In modernen Programmiersprachen (JavaScript, Python, Java) arbeitet man bereits mit Objekten. Hier kann man das Objekt einfach 1:1 in die Datenbank "werfen", ohne es vorher mühsam in Tabellen zu zerlegen (man nennt das Problem Object-Relational Impedance Mismatch).

Flexibilität Wenn man morgen entscheidet, dass Kunden auch ihr "Lieblingsessen" speichern können, fügt man das Feld einfach bei neuen Dokumenten hinzu. Die Datenbank zwingt uns nicht, die gesamte Struktur für Millionen alter Datensätze sofort zu ändern.

Unser Streifzug durch drei NoSQL-Welten:

Column-Family Schnell für Analysen riesiger Datenmengen (Big Data).

Time-Series Spezialisiert auf Zeitstempel und Monitoring (IoT).

Document-Store Flexibel und ideal für Web- und verteilte-Applikationen.

Eine wichtige Ergänzung: "Polyglot Persistence"

Heutzutage nutzt man selten nur *eine* Datenbank. Ein modernes System nutzt oft SQL für die Rechnungen (wegen ACID), MongoDB für das User-Profil (wegen Flexibilität) und Redis (Key-Value) für den schnellen Login-Cache.

Ein wichtiger Punkt für das Verständnis der NoSQL-Landschaft. Während man in der relationalen Welt mit SQL fast überall "zu Hause" ist (egal ob MariaDB, PostgreSQL oder Oracle), gleicht die NoSQL-Welt eher einem Werkzeugkasten mit Spezialwerkzeugen für spezifische Aufgaben.

1. SQL vs. NoSQL: Standardisierung vs. Spezialisierung

Die SQL-Einheitlichkeit

SQL (Structured Query Language) ist seit 1986 standardisiert. Wenn man SQL kann, verstehst man das Grundprinzip fast jeder relationalen Datenbank. Der Fokus liegt auf der Struktur.

Die NoSQL-Vielfalt

In der NoSQL-Welt gibt es keinen universellen Standard. Das liegt daran, dass die Abfragesprache direkt an die Datenstruktur gekoppelt ist:

- Graph-Datenbanken: nutzen Sprachen wie Cypher, um Pfade und Beziehungen zu beschreiben
- Document-Stores: nutzen oft JSON-basierte Abfragen
- Key-Value-Stores: kommen oft mit einfachen Befehlen wie GET und SET aus

2. MQL – Die MongoDB Query Language

Da MongoDB der bekannteste Document-Store ist, schauen wir uns deren Sprache MQL an. Im Gegensatz zu SQL, das wie ein englischer Satz aufgebaut ist (SELECT ... FROM ... WHERE), ist MQL objektorientiert und nutzt die JSON-Syntax. Beispiel: Einen User finden
In SQL und in MQL (MongoDB).

Was fällt uns hier auf?

JSON-Struktur: Die Kriterien stehen in geschweiften Klammern {}.

Operatoren: Statt mathematischer Symbole nutzt MQL oft Schlüsselwörter mit Dollarzeichen, wie \$lt für "less than" (kleiner als) oder \$gte für "greater than or equal".

Methoden-Aufruf: Es fühlt sich eher wie Programmieren in JavaScript an als wie das Schreiben eines Berichts.

```
1 SELECT *  
2 FROM users u  
3 WHERE u.status = "A"  
4 AND u.age < 30;
```

```
1 db.users.find({  
2   status: "A",  
3   age: { $lt: 30 }  
4 })
```

3. Die "Spezialisten-Philosophie"

Dass jede NoSQL-Datenbank ihre eigene Sprache pflegt, hat einen tieferen Grund: Effizienz durch Spezialisierung.

- Redis (Key-Value): Hat Befehle wie INCR (erhöhe einen Wert um 1). Das ist extrem schnell für Counter (z.B. Besucherzahlen), was in SQL viel aufwendiger wäre.
- Neo4j (Graph): Nutzt Symbole wie (p:Person)-[:FRIEND]->(f:Friend), um visuell darzustellen, wie Daten verbunden sind.
- InfluxDB (Time Series): Hatte lange eine SQL-ähnliche Sprache (InfluxQL), ist aber für zeitbasierte Funktionen wie "Gib mir den Durchschnitt der letzten 5 Minuten" optimiert.

Fazit

Wenn man in einem Projekt entscheidet, eine NoSQL-Datenbank einzusetzen, kauft man die Spezialisierung mit ein, muss aber im Team sicherstellen, dass das Know-how für genau diese spezifische Abfragesprache vorhanden ist. Man kann nicht einfach "NoSQL-Entwickler" sein, man ist meistens "MongoDB-Experte" oder "Cassandra-Spezialist".

1. Key-Value Stores (Der Sprinter)

Stell dir uns ein riesiges Schließfach-System vor. Man hat einen Eindeutigen Schlüssel (Key) und dahinter liegt ein Wert (Value). Was in dem Paket (Value) liegt, ist der Datenbank völlig egal – es kann ein Name, eine Zahl oder ein komplettes JSON-Objekt sein.

Das Prinzip:

- Struktur: Eine riesige Liste von Key -> Value Paaren.
- Technik: Meistens liegen diese Daten komplett im RAM (In-Memory), nicht auf der Festplatte.
- Abfrage: Fast ausschließlich über den Key (z. B. GET session:user:123).

Warum nutzt man das?

Caching: Um die langsame SQL-Datenbank zu entlasten. Häufige Anfragen werden im Key-Value Store zwischengespeichert.

Session-Management: Wer ist eingeloggt? Warenkorb-Inhalt während der Sitzung.

Echtzeit-Counter: Live-Besucherzahlen auf einer Website.

Redis als bekanntester Vertreter: Redis ist so schnell, dass es Latenzen im Mikrosekundenbereich bietet.

2. Graph-Datenbanken (Der Netzwerker)

Hier verlassen wir die Tabellen-Logik komplett. In einer Graph-Datenbank sind Beziehungen (Relationships) "First-Class Citizens". Das bedeutet, die Verbindung zwischen zwei Datenpunkten ist genauso wichtig wie die Datenpunkte selbst.

Die Komponenten:

- Nodes (Knoten): Die Objekte (z. B. Person "Max", Produkt "Laptop").
- Edges (Kanten): Die Linien dazwischen (z. B. "ist befreundet mit", "hat gekauft").
- Properties (Eigenschaften): Details an Knoten oder Kanten (z. B. "seit 2020").

Der Vorteil:

In SQL müsste man für eine Abfrage wie "Zeige mir Freunde von Freunden, die das gleiche Produkt gekauft haben" unzählige Joins machen, was die DB in die Knie zwingt. Im Graphen "läuft" die Datenbank einfach die Linien entlang (Graph Traversal) – das ist extrem effizient.

Anwendungsfälle: Soziale Netzwerke (LinkedIn/Facebook), Empfehlungs-Engines (Amazon: "Kunden, die das kauften..."), Betrugserkennung bei Banken.

Bekanntester Vertreter: Neo4j.

Was kann zusammenfassend über NoSQL gesagt werden?

Typ	Fokus	Slogan
Key-Value	Performance	„Ich finde den Wert zum Schlüssel sofort.“
Document	Flexibilität	„Speichere es so, wie es die App braucht.“
Column-Family	Masse & Analyse	„Lies Milliarden Preise in Millisekunden.“
Graph	Beziehungen	„Wer ist mit wem wie verbunden?“
Time-Series	Zeitverlauf	„Was passierte wann in welcher Sekunde?“

Wenn das Projekt...	...dann wähle...	Weil
Finanzdaten, ERP, Lager	Relational (SQL)	ACID-Garantien und Konsistenz hier überlebenswichtig sind.
User-Profile, CMS, Blogs	Document (MongoDB)	sich Anforderungen ständig ändern und Daten oft verschachtelt sind.
Caching, SESSIONs, Warenkorb	Key-Value (Redis)	du extrem Geschwindigkeit im Millisekundenbereich brauchst.
Sozial Network, Betrugsanalyse	Graph (Neo4j)	die Beziehungen zwischen den Daten das eigentliche Wissen sind.
IoT-Sensoren, Server-Monitoring	Time-Series (InfluxDB)	du Milliarden Datenpunkte mit Zeitstempel effizient speichern musst.
Big Data Analytics, Reports	Columnar (Cassandra)	du Summen über Milliarden von Zeilen in Sekunden berechnen willst.



Vielen Dank für eure Aufmerksamkeit.